

# **Project KNOT – Developer & Maintainer Guide**



Team Alpha Coders

Group NO: 19

Date : 25/09/2026

## Table Of Contents

1. System Architecture & Technology Stack
2. Repository & Workspace Organization
3. Database Schema & ER Specifications
4. Environment Setup & Local Installation
5. Core System Logic & Workflows
  - 5.1 Multi-Tier Academic Booking Workflow
  - 5.2 Automated 2-Step Booking Verification Engine
  - 5.3 Conflict-Free Auto-Booking Approval Engine
  - 5.4 Maintenance Ticketing & GIS Dispatch Pipeline
  - 5.5 Master Semester Schedule Importer
6. API Endpoint Reference
  - 6.1 Authentication Endpoints (/api/auth)
  - 6.2 Booking Endpoints (/api/bookings)
  - 6.3 Maintenance & Ticketing Endpoints (/api/tickets)
  - 6.4 System Settings & Rooms Endpoints (/api/settings, /api/halls)
7. Email & Notification Services](#7-email--notification-services)
8. Cloudflare Tunnel & Hosted Server Deployment
  - 8.1 Cloudflare Tunnel Architecture
  - 8.2 cloudflared Daemon Installation & Ingress Setup
  - 8.3 PM2 Production Process Management
  - 8.4 SSL/TLS & Edge Security Configuration
9. Docker Containerization & Multi-Container Stack
10. Testing, Monitoring & Maintenance Procedures

# 1. System Architecture & Technology Stack

**Project KNOT** is architected as a decoupled micro-service platform managed by a central **Gateway Server**. It integrates academic resource reservations with campus maintenance ticketing.

graph TD

Client[Browser / Client SPA] -->|HTTPS Port 443| Cloudflare[Cloudflare Edge Tunnel]  
Cloudflare -->|https://foe.knotpdn.tech| Gateway[Gateway Server Proxy - Port 3000]  
Cloudflare -->|https://backend.knotpdn.tech| StudentAPI[Student & Booking API - Port 5001]

Gateway -->|Frontend Requests| Vite[Vite React Server - Port 5173]  
Gateway -->|/api Backend Requests| StudentAPI  
Gateway -->|/api/maintenance Requests| MaintAPI[Maintenance API - Port 5003]

StudentAPI -->|SQL Pool Queries| DB[(MySQL 8.0 Database - knot\_db)]  
MaintAPI -->|SQL Pool Queries| DB

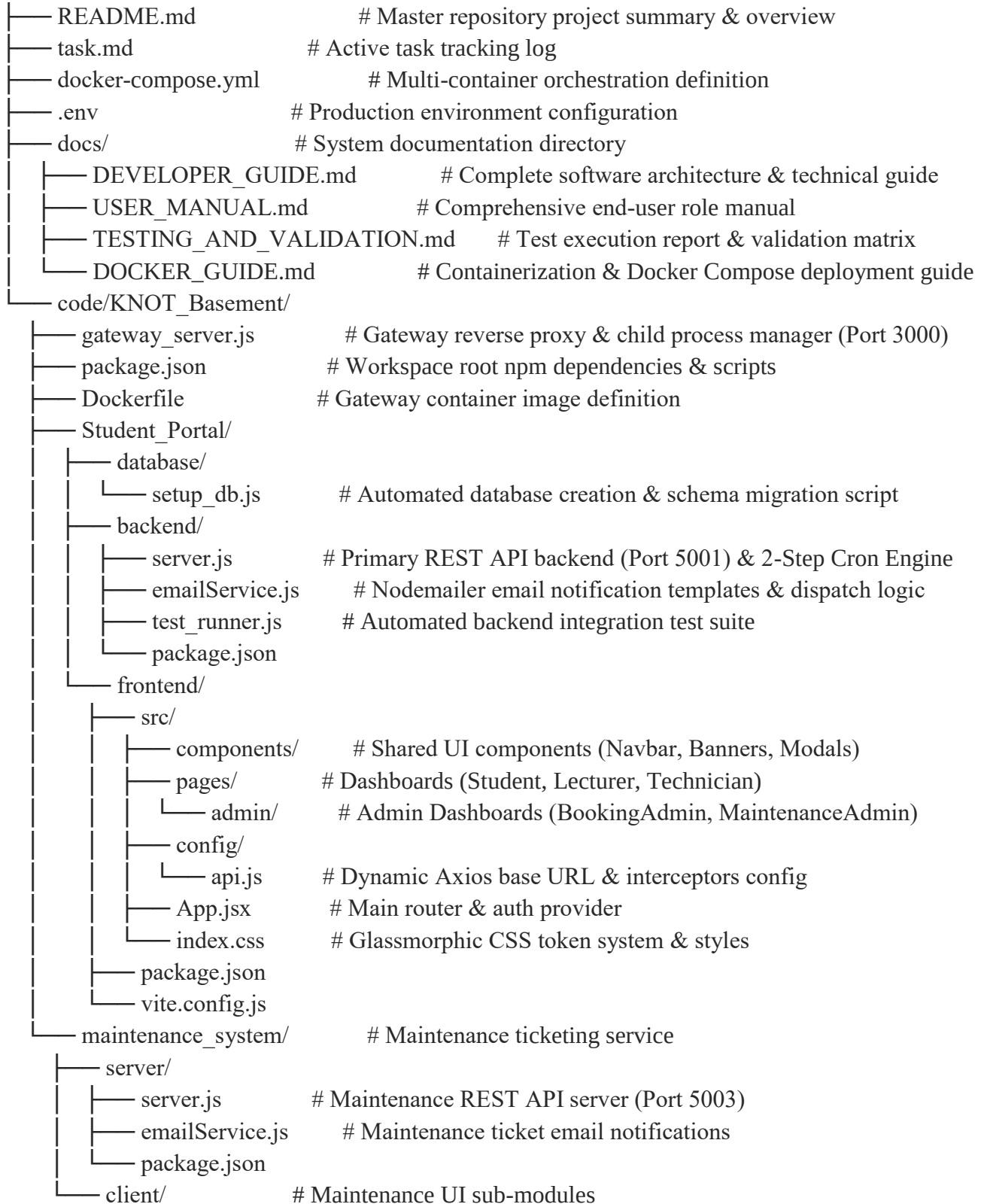
StudentAPI -->|Background Cron - 60s| VerificationEngine[2-Step Verification Engine]  
StudentAPI -->|Async SMTP Dispatch| Email[Nodemailer / SMTP Server]  
MaintAPI -->|Async SMTP Dispatch| Email

## Technology Stack Matrix

| Layer               | Component / Technology                | Specification & Role                                      |
|---------------------|---------------------------------------|-----------------------------------------------------------|
| Frontend Core       | React 18, Vite                        | High-performance SPA with fast HMR                        |
| Routing             | React Router DOM v6                   | Role-guarded client-side routes                           |
| Styling & UI Tokens | Vanilla CSS, Tailwind CSS             | Custom Glassmorphic design tokens & responsive components |
| Map & GIS           | Leaflet, React-Leaflet, Nominatim API | Interactive pin-dropping & reverse geocoding              |
| Backend Core        | Node.js (v18+), Express.js            | REST APIs supporting 50MB Base64 payload limits           |
| Database            | MySQL 8.0 / MariaDB 10.5+             | mysql2/promise with connection pooling                    |
| Background Jobs     | Node.js Interval Timers               | Minute-by-minute 2-Step Verification Cron Worker          |
| Email Service       | Nodemailer                            | Async HTML notification dispatch via SMTP                 |
| Reverse Proxy       | Custom Node.js Gateway & Nginx        | Unified single-port entry point & production ingress      |
| Hosting & Tunnel    | Cloudflare Tunnels (cloudflared)      | Secure zero-trust HTTPS edge proxy to hosted server       |

## 2. Repository & Workspace Organization

e22-co2060-Project-KNOT/



### 3. Database Schema & ER Specifications

The database engine is MySQL 8.0 containing the knot\_db database.

erDiagram

```
users ||--o{ bookings : "creates"
users ||--o{ faults : "reports"
users ||--o{ faults : "assigned to"
rooms ||--o{ bookings : "reserved in"
```

```
users {
    int id PK
    string username UK
    string password
    string name
    string role
    string department
    string email UK
    string phone
    string employee_no
    text address
    longtext avatar
}
```

```
bookings {
    int id PK
    string title
    string time_display
    datetime start_datetime
    datetime end_time
    string status
    int user_id FK
    string assigned_lecturer
    text purpose
    string booking_type
    text rejection_reason
    string verification_status
    datetime verification_prompt_time
    datetime verification_deadline
    boolean verification_notified
}
```

```
faults {
    int id PK
    string title
    text description
    text location
    string status
    string priority
}
```

```

int user_id FK
int assigned_technician_id FK
longtext photo_url
longtext worker_photo
text maintenance_notes
text manager_notes
datetime created_at
datetime resolved_at
}

rooms {
int id PK
string name
int capacity
string type
string status
}

settings {
string setting_key PK
string setting_value
}

```

### 3.1 Table Specifications

#### users Table

Stores authentication accounts, role permissions, and contact profiles.

- **Roles:** Student, Lecturer, booking\_admin, maintenance\_admin, Technician.

| Column Name        | Data Type    | Modifiers                     | Description                             |
|--------------------|--------------|-------------------------------|-----------------------------------------|
| <b>id</b>          | INT          | PRIMARY KEY<br>AUTO_INCREMENT | Unique user account ID                  |
| <b>username</b>    | VARCHAR(255) | UNIQUE NOT NULL               | Login username /<br>Registration ID     |
| <b>password</b>    | VARCHAR(255) | NOT NULL                      | Account password                        |
| <b>name</b>        | VARCHAR(255) | NOT NULL                      | Full display name                       |
| <b>role</b>        | VARCHAR(50)  | NOT NULL                      | Access control role<br>tag              |
| <b>department</b>  | VARCHAR(255) | NULL                          | Faculty department                      |
| <b>email</b>       | VARCHAR(255) | UNIQUE NULL                   | User email address<br>for notifications |
| <b>phone</b>       | VARCHAR(50)  | NULL                          | Contact telephone<br>number             |
| <b>employee_no</b> | VARCHAR(100) | NULL                          | Employee /<br>Registration number       |
| <b>address</b>     | TEXT         | NULL                          | Residential / campus<br>address         |

|               |          |      |                           |
|---------------|----------|------|---------------------------|
| <b>avatar</b> | LONGTEXT | NULL | Base64 profile photo data |
|---------------|----------|------|---------------------------|

## bookings Table

Stores hall reservations, schedule times, approval statuses, and 2-step verification states.

| Column Name                     | Data Type    | Modifiers                     | Description                                                          |
|---------------------------------|--------------|-------------------------------|----------------------------------------------------------------------|
| <b>id</b>                       | INT          | PRIMARY KEY<br>AUTO_INCREMENT | Unique booking ID                                                    |
| <b>title</b>                    | VARCHAR(255) | NOT NULL                      | Reserved room name (e.g. EOE Hall)                                   |
| <b>time_display</b>             | VARCHAR(255) | NOT NULL                      | Human-readable schedule string                                       |
| <b>start_datetime</b>           | DATETIME     | NULL                          | Structured reservation start timestamp                               |
| <b>end_time</b>                 | DATETIME     | NULL                          | Structured reservation end timestamp                                 |
| <b>status</b>                   | VARCHAR(50)  | DEFAULT 'Pending'             | Workflow status (Approved, Pending AR, Pending, Rejected, Cancelled) |
| <b>user_id</b>                  | INT          | FK -> users(id)               | ID of booker                                                         |
| <b>assigned_lecturer</b>        | VARCHAR(255) | NULL                          | Endorsing lecturer name                                              |
| <b>purpose</b>                  | TEXT         | NULL                          | Booking purpose / course details                                     |
| <b>booking_type</b>             | VARCHAR(50)  | DEFAULT 'AR Office'           | Request workflow path                                                |
| <b>rejection_reason</b>         | TEXT         | NULL                          | Rejection feedback from admin/lecturer                               |
| <b>verification_status</b>      | VARCHAR(50)  | DEFAULT 'unrequired'          | 2-Step state (unrequired, pending, verified, expired)                |
| <b>verification_prompt_time</b> | DATETIME     | NULL                          | Time when 2-step prompt activates                                    |
| <b>verification_deadline</b>    | DATETIME     | NULL                          | Verification confirmation deadline                                   |
| <b>verification_notified</b>    | BOOLEAN      | DEFAULT FALSE                 | Flag tracking email prompt dispatch                                  |

## faults Table

Tracks reported maintenance issues, GIS coordinates, photos, and technician assignments.

| Column Name                   | Data Type    | Modifiers                     | Description                                |
|-------------------------------|--------------|-------------------------------|--------------------------------------------|
| <b>id</b>                     | INT          | PRIMARY KEY<br>AUTO_INCREMENT | Maintenance ticket ID                      |
| <b>title</b>                  | VARCHAR(255) | NOT NULL                      | Fault title / location tag                 |
| <b>description</b>            | TEXT         | NULL                          | Detailed fault report                      |
| <b>location</b>               | TEXT         | NULL                          | Geographic coordinates + address text      |
| <b>status</b>                 | VARCHAR(50)  | DEFAULT 'Open'                | Ticket state (Open, In Progress, Resolved) |
| <b>priority</b>               | VARCHAR(50)  | DEFAULT 'Medium'              | Urgency (Low, Medium, High, Urgent)        |
| <b>user_id</b>                | INT          | FK -> users(id)               | Ticket reporter ID                         |
| <b>assigned_technician_id</b> | INT          | FK -> users(id)               | Assigned technician worker ID              |
| <b>photo_url</b>              | LONGTEXT     | NULL                          | Base64 evidence photo from reporter        |
| <b>worker_photo</b>           | LONGTEXT     | NULL                          | Base64 proof-of-work photo from technician |
| <b>maintenance_notes</b>      | TEXT         | NULL                          | Work resolution log from technician        |
| <b>manager_notes</b>          | TEXT         | NULL                          | Directives from maintenance admin          |
| <b>created_at</b>             | DATETIME     | DEFAULT<br>CURRENT_TIMESTAMP  | Ticket creation timestamp                  |
| <b>resolved_at</b>            | DATETIME     | NULL                          | Resolution completion timestamp            |

rooms Table

Master catalog of standardized faculty lecture halls, drawing offices, and labs.

| Column Name | Data Type    | Modifiers                     | Description             |
|-------------|--------------|-------------------------------|-------------------------|
| id          | INT          | PRIMARY KEY<br>AUTO_INCREMENT | Room ID                 |
| name        | VARCHAR(255) | NOT NULL                      | Standard room name      |
| capacity    | INT          | DEFAULT 30                    | Seating capacity        |
| type        | VARCHAR(50)  | DEFAULT 'Lecture Hall'        | Room classification     |
| status      | VARCHAR(50)  | DEFAULT 'Available'           | Room operational status |

settings Table

Key-value store for global platform flags.

| Setting Key  | Allowed Values   | Description                                                           |
|--------------|------------------|-----------------------------------------------------------------------|
| auto_booking | 'true' / 'false' | Toggle instant automated booking approvals for conflict-free requests |

## 4. Environment Setup & Local Installation

### 4.1 Prerequisites

- **Node.js:** v18.0.0 or higher
- **npm:** v9.0.0 or higher
- **MySQL:** v8.0 or MariaDB v10.5+

### 4.2 Master Environment Variables (.env)

Create or verify .env in the root workspace directory or code/KNOT\_Basement/.env:

```
# Database Credentials
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=knot_db
DB_PORT=3306
MYSQL_ROOT_PASSWORD=your_mysql_password
MYSQL_DATABASE=knot_db

# Security & Tokens
JWT_SECRET=knot_super_secret_jwt_key_2026

# Email Dispatcher Credentials (SMTP)
EMAIL_USER=minhaj.dssc2@gmail.com
EMAIL_PASS=wvvnkowdrbnfetva
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=minhaj.dssc2@gmail.com
SMTP_PASS=wvvnkowdrbnfetva

# Application Service Ports
PORT=5001
MAINTENANCE_PORT=5003
GATEWAY_PORT=3000
FRONTEND_PORT=5173

# Base Domain URLs
FRONTEND_URL=https://foe.knotpdn.tech
BACKEND_URL=https://backend.knotpdn.tech
VITE_API_URL=https://backend.knotpdn.tech
```

### 4.3 Database Schema Initialization

Run the database setup script to create tables and seed demo accounts:

```
node code/KNOT_Basement/Student_Portal/database/setup_db.js
```

#### 4.4 Launching System via Gateway Server

The **Gateway Server** starts all child micro-services and serves as a unified proxy hub:

```
node code/KNOT_Basement/gateway_server.js
```

Access the application locally at **http://localhost:3000**

#### 4.5 Demo Access Accounts Matrix

| Role               | Username  | Password  | Email Address             |
|--------------------|-----------|-----------|---------------------------|
| Student            | e22237    | 1234      | e22237@eng.pdn.ac.lk      |
| Lecturer           | lecturer1 | 1234      | lecturer1@eng.pdn.ac.lk   |
| Booking Admin (AR) | bookadmin | adminpass | ar.office@eng.pdn.ac.lk   |
| Maintenance Admin  | admin     | adminpass | maint.admin@eng.pdn.ac.lk |
| Technician         | alex      | 1234      | alex.tech@eng.pdn.ac.lk   |

## 5. Core System Logic & Workflows

### 5.1 Multi Tier Academic Booking Workflow

sequenceDiagram

autonumber

actor S as Student / Booker

actor L as Endorsing Lecturer

actor A as Booking Admin (AR Office)

participant DB as MySQL DB

alt Student Reservation Flow

S->>DB: POST /api/bookings (status='Pending', assigned\_lecturer='Dr. Smith')

DB-->>L: Visible on Lecturer Endorsement Dashboard

L->>DB: PUT /api/bookings/:id/status (status='Pending AR' or 'Rejected')

DB-->>A: Visible in AR Office Approval Queue

A->>DB: PUT /api/bookings/:id/status (status='Approved' or 'Rejected')

else Lecturer Direct Reservation Flow

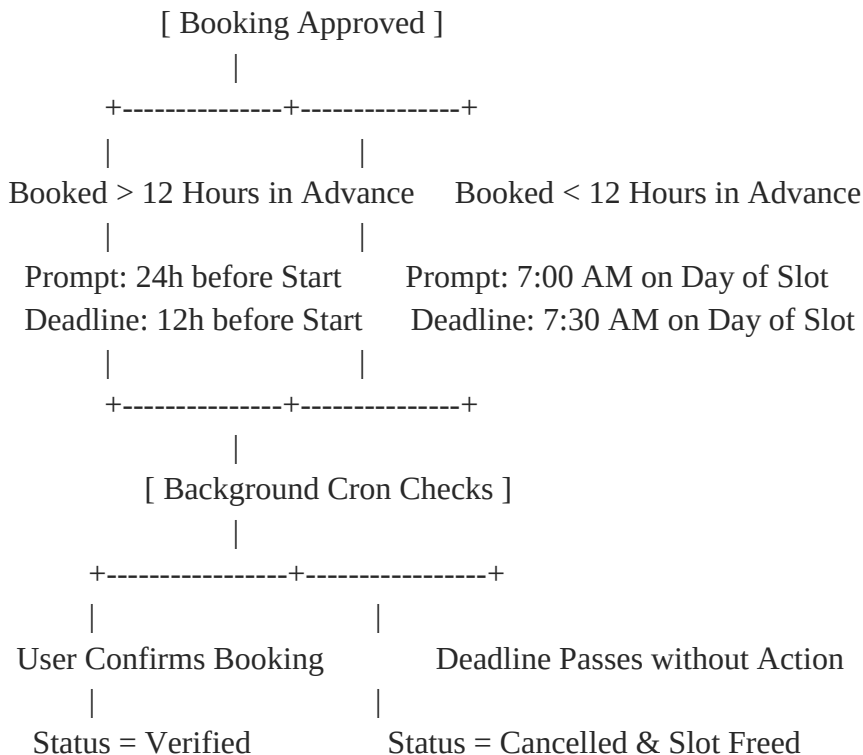
S->>DB: POST /api/bookings (status='Pending AR')

A->>DB: PUT /api/bookings/:id/status (status='Approved')

end

### 5.2 Automated 2-Step Booking Verification Engine

To eliminate empty halls caused by no-shows, KNOT enforces an automated 2-step verification protocol.



**Cron Routine:** processBooking2StepVerifications() executes every **60 seconds** in Student\_Portal/backend/server.js.

**Notification Dispatch:** Sends email prompts when verification opens and notifies bookers if bookings expire due to missed deadlines.

### 5.3 Conflict-Free Auto-Booking Approval Engine

- Controlled by auto\_booking in settings.
- When set to 'true', incoming reservation requests undergo automatic overlap checks against all existing Approved bookings for the same room. If no schedule overlap occurs, the request is immediately marked Approved.

### 5.4 Maintenance Ticketing & GIS Dispatch Pipeline

flowchart LR

```
A[User Reports Fault] -->|Pin Drop + Photo| B[(knot_db Faults Table)]
B --> C[Maintenance Admin Review]
C -->|Assign Technician & Priority| D[Technician Task Queue]
D --> E[Field Work Execution]
E -->|Proof Photo + Log| F[Mark Ticket Resolved]
F --> G[Reporter Email Notification]
```

### 5.5 Master Semester Schedule Importer

- **Purpose:** Meets the Dean's requirement for bulk timetable ingestion.
- **CSV Ingestion:** Accepts timetable CSV files containing Room, Day, Start Time, End Time, and Course/Lecturer.
- **Validation:** Normalizes hall names against the 10 standard rooms (EOE Hall, DO1, DO2, LH01, LH02, Seminar Room A, Seminar Room B, Computer Lab 01, Computer Lab 02, Electronics Lab).
- **Conflict Handling:** Checks existing database schedules, skips conflicting slots, and populates recurring weekly reservations.

## 6. API Endpoint Reference

### 6.1 Authentication Endpoints (/api/auth)

#### POST /api/auth/login

Authenticates user credentials and returns JWT bearer token + user profile.

- **Request Body:**

```
{
  "username": "e22237",
  "password": "1234"
}
```

- **Response 200 OK`:**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "username": "e22237",
    "name": "Minhaj Ali",
    "role": "Student",
    "email": "e22237@eng.pdn.ac.lk"
  }
}
```

#### GET /api/auth/profile

Fetches authenticated user profile details.

- **Headers:** Authorization: Bearer <token>

#### PUT /api/auth/profile

Updates profile information (name, email, phone, employee number, address, avatar).

### 6.2 Booking Endpoints (/api/bookings)

#### GET /api/bookings

Retrieves room reservations based on role permissions and filters.

#### POST /api/bookings

Submits a new room reservation request.

- **Request Body:**

```
{
  "title": "EOE Hall - Engineering South",
  "time_display": "Tomorrow, 10:00 AM - 12:00 PM",
  "start_datetime": "2026-09-26 10:00:00",
  "end_time": "2026-09-26 12:00:00",
  "purpose": "CO2060 Lecture",
  "assigned_lecturer": "Dr. Smith"
}
```

**PUT /api/bookings/:id/status**

Updates booking approval status (Approved, Pending AR, Rejected, Cancelled).

**GET /api/bookings/verifications/:userId**

Retrieves pending 2-step verification prompts for a user.

**PUT /api/bookings/:id/verify**

Confirms or cancels a 2-step verification prompt.

**POST /api/bookings/bulk-import**

Parses and imports master semester timetable CSV files.

**6.3 Maintenance & Ticketing Endpoints (/api/tickets)****GET /api/tickets**

Returns all maintenance tickets (with optional priority/status filters).

**POST /api/tickets**

Creates a new maintenance ticket with GIS pin coordinates and Base64 evidence photo.

- **Request Body:**

```
{
  "title": "Projector Fault - Room 1",
  "description": "HDMI port damaged",
  "location": "{\"lat\":7.2543,\"lng\":80.5921,\"address\":\"Faculty of Engineering\"}",
  "priority": "High",
  "photo_url": "data:image/png;base64,..."
}
```

**PUT /api/tickets/:id**

Updates ticket details, assigned technician ID, worker notes, or proof-of-work photo.

**GET /api/tickets/technician/:techId**

Fetches tasks assigned to a specific field technician.

**6.4 System Settings & Rooms Endpoints (/api/settings, /api/halls)****GET /api/halls**

Returns master room list and operational statuses.

**GET /api/settings**

Returns global system settings.

**PUT /api/settings**

Updates global setting key-value pairs (e.g. auto\_booking).

## 7. Email & Notification Services

Email notifications are dispatched asynchronously via emailService.js using Nodemailer.

| Trigger Event              | Recipient                    | Email Template Function                        |
|----------------------------|------------------------------|------------------------------------------------|
| Fault Ticket Created       | Reporter & Maintenance Admin | sendTicketCreatedNotification()                |
| Technician Assigned        | Assigned Worker              | sendTechnicianAssignmentNotification()         |
| Ticket Resolved            | Ticket Reporter              | sendTicketResolvedNotification()               |
| 2-Step Verification Prompt | Booker                       | sendBookingVerificationPromptNotification()    |
| Booking Confirmed          | Booker                       | sendBookingVerificationConfirmedNotification() |
| Booking Cancelled (Expiry) | Booker                       | sendBookingVerificationExpiredNotification()   |

## 8. Cloudflare Tunnel & Hosted Server Deployment

To expose the locally hosted server securely over HTTPS without configuring complex firewall port forwarding or static public IPs, **Project KNOT** utilizes **Cloudflare Tunnels** (`cloudflared`).

graph LR

User[End User Browser] -->|HTTPS| CF[Cloudflare Edge Network]

CF -->|Encrypted Tunnel Protocol| Cloudflared[cloudflared Daemon on Host Server]

Cloudflared -->|http://localhost:3000| Gateway[Node.js Gateway Hub]

Cloudflared -->|http://localhost:5001| StudentAPI[Student Backend API]

### 8.1 Public Domain Mapping

- **Frontend Application:** `https://foe.knotpdn.tech` → Proxied to `http://localhost:3000` (Gateway Hub) or `http://localhost:5173`
- **Backend API:** `https://backend.knotpdn.tech` → Proxied to `http://localhost:5001` (Student & Booking REST API)

### 8.2 cloudflared Daemon Installation & Ingress Setup

#### Step 1: Install cloudflared on Host Server

- **macOS (Homebrew):**

```
brew install cloudflared
```

- **Linux (Ubuntu/Debian):**

```
curl -L --output cloudflared.deb
```

```
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb
```

```
sudo dpkg -i cloudflared.deb
```

#### Step 2: Authenticate & Create Tunnel

```
cloudflared tunnel login
```

```
cloudflared tunnel create knot-tunnel
```

This generates a tunnel ID and credentials JSON file in `~/.cloudflared/`.

### Step 3: Configure config.yml

Create a configuration file at ~/.cloudflared/config.yml:

```
tunnel: <YOUR-TUNNEL-UUID>
credentials-file: /root/.cloudflared/<YOUR-TUNNEL-UUID>.json

ingress:
  - hostname: foe.knotpdn.tech
    service: http://localhost:3000
  - hostname: backend.knotpdn.tech
    service: http://localhost:5001
  - service: http_status:404
```

### Step 4: Map DNS Routes & Start Tunnel Daemon

```
cloudflared tunnel route dns knot-tunnel foe.knotpdn.tech
cloudflared tunnel route dns knot-tunnel backend.knotpdn.tech
```

```
# Run as background service
sudo cloudflared service install
sudo systemctl start cloudflared
```

## 8.3 PM2 Production Process Management

On the host server, use **PM2** to keep backend micro-services and the Gateway server running continuously.

```
# Install PM2 globally
npm install -g pm2

# Start Gateway Server
pm2 start code/KNOT_Basement/gateway_server.js --name "knot-gateway"

# Save process list for auto-restart on system boot
pm2 save
pm2 startup
```

Useful PM2 Commands:

```
pm2 status      # Check running status of services
pm2 logs knot-gateway # View real-time output logs
pm2 restart all  # Restart all processes
```

## 8.4 SSL/TLS & Edge Security Configuration

- SSL termination is managed automatically by Cloudflare's Edge Certificates.
- Set Cloudflare SSL/TLS Encryption mode to **Full** or **Flexible** in the Cloudflare Dashboard.

## 9. Docker Containerization & Multi-Container Stack

The entire platform can be deployed in containerized environments using Docker and Docker Compose.

```
# Build and launch multi-container stack in detached mode
docker-compose up --build -d
```

### Container Stack Specification

| Container Name                  | Base Image     | Exposed Port | Purpose                                            |
|---------------------------------|----------------|--------------|----------------------------------------------------|
| <b>knot-db</b>                  | mysql:8.0      | 3306         | Persistent MySQL database container                |
| <b>knot-student-backend</b>     | node:18-alpine | 5001         | Express REST API & 2-step verification cron engine |
| <b>knot-maintenance-backend</b> | node:18-alpine | 5003         | Maintenance ticketing backend                      |
| <b>knot-frontend</b>            | nginx:alpine   | 80, 5173     | Production React SPA served via Nginx              |
| <b>knot-gateway</b>             | node:18-alpine | 3000         | Gateway reverse proxy hub                          |

## 10. Testing, Monitoring & Maintenance Procedures

### 10.1 Running Backend Automated Integration Tests

Execute the backend integration test runner:

```
node code/KNOT_Basement/Student_Portal/backend/test_runner.js
```

### 10.2 Database Backup & Disaster Recovery

- Backup Database:

```
mysqldump -u root -p knot_db > knot_db_backup.sql
```

- Restore Database:

```
mysql -u root -p knot_db < knot_db_backup.sql
```

### 10.3 Inspecting Real-Time Logs

- Gateway Server: Inspect terminal standard output or PM2 logs (pm2 logs).
- Docker Stack: docker-compose logs -f backend.